`find_element` to search the remaining elements ⓬. As already mentioned, you need to wait for all threads to finish before you check the result, because there might not be *any* matching elements. You do this by enclosing the thread launching-and-joining code in a block ⓾, so all threads are joined when you check the flag to see whether a match was found ⓭. If a match was found, you can get the result or throw the stored exception by calling `get()` on the `std::future<Iterator>` you can get from the promise ⓮.

Again, this implementation assumes that you're going to be using all available hardware threads or that you have some other mechanism to determine the number of threads to use for the up-front division of work between threads. Just as before, you can use `std::async` and recursive data division to simplify your implementation, while using the automatic scaling facility of the C++ Standard Library. An implementation of `parallel_find` using `std::async` is shown in the following listing.

Listing 8.10. An implementation of a parallel find algorithm using `std::async`

```
template<typename Iterator,typename MatchType>                          ❶
Iterator parallel_find_impl(Iterator first,Iterator last,MatchType match,
                            std::atomic<bool>& done)
{
    try
    {
        unsigned long const length=std::distance(first,last);
        unsigned long const min_per_thread=25;          ❷
        if(length<(2*min_per_thread))                   ❸
        {
            for(;(first!=last) && !done.load();++first)     ❹
            {
                if(*first==match)
                {
                    done=true;      ❺
                    return first;
                }
            }
            return last;    ❻
```